Meilenstein 03 – AuthKit: Docker, CI/CD mit GitHub Actions und Veröffentlichung auf ghcr.io

Feld	Wert
Projekt	Overex
Zeitraum	2026-10-09 – 2026-10-10
Dokumentiert am	2026-10-10 20:28 CEST
Repository / Service	`authkit` (github.com/Umex10/authkit), Backend `apps/backend` (künftiger `auth-service` von Overex)
Thema	Backend-Image mit Docker bauen und lokal mit Keys starten, drei GitHub-Actions-Workflows (Tests, Image-Build, Veröffentlichung auf `ghcr.io`)
Commits	5 (`119a0e9` .. `edd0e90`), Branch `feature/docker`, über PR #1 nach `main` gemergt (Merge-Commit `edd0e90`)
Zeitaufwand	nicht erfasst
Bezug	Projektarbeit (5. Semester, Abgabe Jänner 2027) und Grundlage für die Bachelorarbeit (6. Semester), insbesondere für den Kubernetes-Teil

Ein großer Teil dieses Meilensteins war Lernen: Ich kannte Docker und CI aus meiner Arbeit mit Next.js und NestJS, aber nicht die Java-Seite (Maven, Jar, JDK). Die Grundlagen, die ich mir dabei erarbeitet habe, stehen ausführlich im **Anhang** am Ende dieses Dokuments.

Ziel

Für die Bachelorarbeit wird Overex auf Kubernetes laufen. Kubernetes startet Container aus Images, die in einer Registry liegen. Darum braucht AuthKit:

- ein Docker-Image, das ohne Geheimnisse gebaut wird und die Keys erst beim Start bekommt,
- eine CI, die bei jedem Push prüft, ob die Tests grün sind und ob sich das Image bauen lässt,
- eine automatische Veröffentlichung des Images auf `ghcr.io`, sobald Code nach `main` gemergt wird.

Außerdem wollte ich Docker so gut verstehen, dass der Einstieg in Kubernetes leichter wird.

Ausgangslage

- Es gab schon eine Multi-Stage-`Dockerfile` (JDK zum Bauen, JRE zum Ausführen) und eine `docker-compose.yml` mit Postgres, Backend und Swagger UI.
- Der Backend-Container ließ sich seit Meilenstein 01 nicht mehr starten: Die Keys sind absichtlich nicht im Image, und `docker-compose.yml` gab sie dem Container auch nicht von außen.
- Es gab keine Workflows unter `.github/workflows/`.
- Auf `main` lag noch der alte HS256-Stand. Die Arbeit aus Meilenstein 01 und 02 war nur auf Feature-Branches.

Umsetzung (nach Commits)

Die Zeitpunkte der Commits stehen im Commit-Verlauf am Ende, die Zeitpunkte der Workflow-Läufe in „Tests & Verifikation“.

1. **`5c35460`** – Keys per Bind Mount in den Backend-Container gegeben (`./apps/backend/keys:/app/keys:ro` in `docker-compose.yml`). Vorher habe ich die `Dockerfile` Zeile für Zeile durchgearbeitet und das Image lokal gebaut (396 MB). Der erste Start scheiterte mit `Permission denied`, weil der Container als uid 1001 läuft und `private.pem` nur für mich lesbar war. Gelöst mit `chmod 644` auf den Dev-Key. Dazu Notizen zu JDK, JRE und JVM in der `Dockerfile`.
2. **`e317fde`** – Erster Workflow `docker-ci`: baut das Image bei jedem Push auf einen Branch außer `main`, ohne Push, mit Layer-Cache im GitHub-Cache.
3. **`8641c70`** – Zwei weitere Workflows:

- `docker-publish` : baut auf `main` (oder per Knopf), meldet sich mit dem `GITHUB_TOKEN` bei `ghcr.io` an und pusht das Image mit den Tags `latest` und `sha-<commit>`
- `backend-verify-ci` : `./mvnw verify` auf jedem Branch, mit Maven-Cache und den Reports als Artifact. Der Integrationstest ist vorübergehend ausgenommen, weil er noch lokale Key-Dateien braucht.

4. **`0a1ea83`** – Jede Anweisung der `Dockerfile` und jedes Feld von `docker-ci` mit englischen Kommentaren versehen. Die Kommentare der beiden anderen Workflows waren schon in `8641c70` enthalten.
5. **`edd0e90`** – PR #1 `feature/docker → main` mit „Create a merge commit“ gemergt. Damit kamen auch die Arbeiten aus Meilenstein 01 und 02 nach `main` . Auf `main` liefen `backend-verify-ci` und `docker-publish` grün, das Image erschien unter „Packages“.

Entscheidungen

Entscheidung	Verworfene Alternativen	Begründung
Jar wird in der `Dockerfile` gebaut (Multi-Stage)	Jar auf der CI-Maschine bauen, `Dockerfile` kopiert nur noch die fertige Jar	Die `Dockerfile` bleibt in sich geschlossen: Zum Bauen braucht man nur Docker und den Code. Kein Risiko, eine alte Jar aus `target/` ins Image zu packen.
Keys per Bind Mount zur Laufzeit	Keys ins Image kopieren	Das Image liegt in einer Registry. Wer es herunterlädt, könnte den privaten Key auslesen und eigene Tokens ausstellen.
`chmod 644` auf den Dev-Key	`user: "1000:1000"` in `docker-compose.yml`	Der Container läuft weiter wie in Produktion als uid 1001. Es ist ein reiner Entwicklungsschlüssel auf einem Rechner mit einem Benutzer.
Backend über `docker compose` starten	`docker run` von Hand	Das Backend braucht Postgres im selben Netzwerk. Compose startet und verbindet beides.
Drei getrennte Workflows	Tests als zweiter Job in `docker-ci`	Die Auslöser sind verschieden: Tests auf allen Branches, `docker-ci` nicht auf `main` , `docker-publish` nur auf `main` . Der Auslöser gilt pro Workflow, nicht pro Job.
Buildx mit `build-push-action` und Cache im GitHub-Cache (`type=gha`)	einfaches `docker build`	Die Maschine ist bei jedem Lauf leer. Nur Buildx kann Layer in einem externen Cache ablegen und wieder laden.
`mode=max`	Standard `mode=min`	Ohne `max` würden nur die Layer des fertigen Images gespeichert. Die teure Schicht mit den Maven-Abhängigkeiten liegt aber in der Build-Stage.
Tags `latest` und `sha-<commit>`	Versionsnummern (`1.0.0`)	AuthKit hat noch keinen Release-Ablauf (`pom.xml` steht auf `0.0.1-SNAPSHOT`). Die Commit-Nummer entsteht automatisch und zeigt genau, aus welchem Stand das Image gebaut wurde.
Anmeldung mit dem automatischen `GITHUB_TOKEN` (`packages: write`)	eigener Personal Access Token als Secret	Kein Geheimnis muss angelegt oder gepflegt werden, der Token gilt nur für den einen Lauf.
`concurrency` : Abbruch bei CI, Warteschlange bei Publish	überall abbrechen	Ein abgebrochener Push könnte ein halb hochgeladenes Image hinterlassen, und jeder Merge soll sein eigenes Image bekommen.
Integrationstest per `-Dtest='!…/integration/**'` im Workflow ausnehmen	`@Disabled` im Testcode; Keys in der CI erzeugen	Der Testcode bleibt unverändert, sodass mein Stash ohne Konflikt zurückkommt. Zurücknehmen ist eine Zeile.
`if: ${{ !cancelled() }}` beim Upload	`if: always()`	`always()` läuft auch bei abgebrochenen Läufen. Halbe Reports eines abgebrochenen Laufs braucht niemand.

„Create a merge commit“ für PR #1	„Squash and merge“, „Rebase and merge“	Die Commits sind sauber, und meine Doku verweist auf Commit-Hashes. Squash würde die Commits zusammenfassen, Rebase würde ihnen neue Hashes geben.
Docker Engine direkt auf Linux	Docker Desktop (mit VM)	Container laufen direkt auf meinem Kernel, so wie später auf den Kubernetes-Knoten. Die VM brächte für eigene und offizielle Images kaum Sicherheitsgewinn.
Aktuelle Hauptversionen der Actions (`checkout@v7` , `setup-buildx-action@v4` , `build-push-action@v7` , `login-action@v4` , `metadata-action@v6` , `setup-java@v6` , `upload-artifact@v7`)	ältere Versionen aus Beispielen	Am 2026-10-10 über die GitHub-API geprüft.

Neu gelernt

- **Maven**: Build-Tool und Paketmanager in einem. Es lädt die Abhängigkeiten aus dem `pom.xml` , kompiliert, testet und packt die Jar. Die Phasen laufen in fester Reihenfolge, jeder Aufruf nimmt alle Phasen davor mit.
- **`./mvnw`** : ein Skript im Repo, das die in `.mvn/wrapper/maven-wrapper.properties` festgelegte Maven-Version (3.9.14) lädt. So bauen alle mit derselben Version. Mein installiertes `mvn` ist 3.8.7.
- **Dependency und Plugin**: Dependencies sind Bibliotheken, die mein Code importiert (Zutaten). Plugins sind Werkzeuge, die Maven beim Bauen benutzt (Küchengeräte).
- **`~/.m2`** : Alle Abhängigkeiten liegen zentral in `~/.m2/repository` . Es gibt kein `node_modules` pro Projekt.
- **Fat Jar**: Maven allein baut eine Jar nur mit meinem Code. Das Spring-Boot-Plugin baut daraus eine Jar mit allen 106 Bibliotheken und einem Startprogramm, die man mit `java -jar` starten kann.
- **JDK, JRE, JVM**: Die JVM führt Bytecode aus. Das JRE ist JVM plus Standardbibliothek. Das JDK ist JRE plus Compiler. Gebaut wird mit dem JDK, ausgeführt mit dem JRE.
- **Image und Container**: Ein Image ist ein Stapel unveränderlicher Dateisystem-Schichten. Ein Container ist ein laufender, abgeschotteter Prozess daraus.
- **Container unter Linux**: ein normaler Prozess, dem der Kernel mit Namespaces eine eigene Sicht und mit cgroups Grenzen gibt. Er nutzt den Kernel des Rechners. Windows und macOS brauchen dafür eine Linux-VM.
- **Layer-Cache**: Docker merkt sich das Ergebnis jeder Anweisung. Ab der ersten Änderung wird alles darunter neu gebaut. Darum steht `dependency:go-offline` vor `COPY src` .
- **`ENTRYPOINT` und `CMD`** : `ENTRYPOINT` ist das Programm, das immer läuft. `CMD` ist der Standard dahinter, den man beim Start ersetzen kann.
- **Bind Mount und Volume**: Ein Bind Mount bindet einen Ordner meines Rechners ein. Ein benanntes Volume ist ein Speicher, den Docker verwaltet.
- **Linux-Rechte**: `r` =4, `w` =2, `x` =1, je für Besitzer, Gruppe und alle anderen. `644` heißt: ich lese und schreibe, alle anderen lesen.
- **GitHub Actions**: Ein Workflow hat `on` (wann), `jobs` (auf welcher Maschine) und `steps` . Ein Step ist entweder ein Befehl (`run`) oder eine fertige Action (`uses` , Einstellungen über `with`).
- **Registry**: ein Server, der Pakete eines Formats aufbewahrt und ausgibt. `ghcr.io` ist die Docker-Registry von GitHub und Teil von GitHub Packages.

Tests & Verifikation

Test	Ergebnis
`docker build -t authkit-backend:local .`	erfolgreich, Image 396 MB, Benutzer `authkit` , `ENTRYPOINT [java -jar app.jar]`
`docker compose up --build backend` ohne passende Rechte auf `private.pem`	Abbruch: `FileNotFoundException: keys/private.pem (Permission denied)`
dasselbe nach `chmod 644 keys/private.pem`	„Started BackendApplication in 4.487 seconds“, Datenbank `jdbc:postgresql://db:5432/authkit`
Probe-Build der kommentierten `Dockerfile`	erfolgreich
`./mvnw test -Dtest='!com/authkit/backend/integration/**'` lokal	45 Tests grün (Unit und Slice)

`docker-ci` auf `feature/docker`, `e317fde` (leerer Cache)	grün, 15:20:53–15:22:48 CEST (ca. 2 min)
`docker-ci` auf `feature/docker`, `0a1ea83` (Cache, nur Kommentare geändert)	grün, 19:41:08–19:41:25 CEST (17 s)
`backend-verify-ci` auf `feature/docker`, `0a1ea83`	grün, 49 s
`backend-verify-ci` auf `main`, `edd0e90`	grün, 50 s
`docker-publish` auf `main`, `edd0e90`	grün, 20:03:45–20:06:02 CEST (2 min 17 s)
Image `authkit-backend` unter „Packages“	laut User vorhanden. Über die API nicht prüfbar, weil dem lokalen GitHub-Zugang das Recht `read:packages` fehlt.

Die Zeiten der Workflow-Läufe stammen aus `gh run list`.

Probleme & Lösungen

- **Problem:** Backend-Container brach mit `Permission denied` ab. → **Lösung:** Die Datei war `600` (nur Besitzer 1000), der Container läuft als 1001. `chmod 644` auf den Dev-Key.
- **Problem:** Der Integrationstest würde in der CI rot, weil es dort keine Key-Dateien gibt. → **Lösung:** vorübergehend über `-Dtest='!com/authkit/backend/integration/**'` ausgenommen, mit `TEMPORARY`-Kommentar im Workflow.
- **Problem:** Kommentare standen innerhalb des Blocks `tags: |` der `metadata-action` und wären dort als Text behandelt worden. → **Lösung:** Kommentare über `tags: |` verschoben und den Inhalt des Blocks geprüft.
- **Problem:** Die Beispiele enthielten veraltete Versionen der Actions (`actions/checkout@v4`). → **Lösung:** Hauptversionen über die GitHub-API geprüft und aktualisiert.
- **Problem:** Mein GitHub-Name `Umex10` enthält einen Großbuchstaben, Image-Namen dürfen das nicht. → **Lösung:** `metadata-action` wandelt den Namen in Kleinbuchstaben um (`ghcr.io/umex10/authkit-backend`).
- **Problem:** Ich dachte, das veröffentlichte Image sei fehlerhaft, weil die Keys fehlen. → **Lösung:** Das ist Absicht. Wer das Image startet, muss die Keys mitgeben (Compose: Bind Mount, Kubernetes: Secret). Ohne Keys bricht die App sofort mit klarer Meldung ab.
- **Problem:** PR #1 enthielt 15 Commits statt nur den Docker-Teil, weil `feature/docker` von `feature/rs256-jwks` abgezweigt war und dieser Branch nie gemergt wurde. → **Lösung:** bewusst als ein PR gemergt, Titel und Beschreibung decken alles ab.

Offene Punkte / Nächste Schritte

- **Integrationstest fertigstellen** (siehe Meilenstein 02) und danach die Ausnahme `-Dtest=...` aus `backend-verify-ci.yml` entfernen.
- **Sichtbarkeit des Pakets prüfen** (öffentlich oder privat). Für Kubernetes braucht ein privates Image Zugangsdaten im Cluster.
- **Kein Image mit roten Tests veröffentlichen:** `backend-verify-ci` und `docker-publish` laufen unabhängig. Der Publish-Workflow sollte vorher selbst die Tests ausführen (zum Beispiel zwei Jobs mit `needs`).
- **Auslöser `pull_request`** ergänzen, damit Workflows den Stand nach dem Merge schon im PR prüfen.
- **Releases mit Versionsnummern**, sobald die RS256-Umstellung komplett ist (zum Beispiel `v1.0.0`): Tag-Auslöser und `type=semver` in der `metadata-action`.
- `docs/BACKEND_FLOW.md` und `docs/ARCHITECTURE.md` beschreiben noch den alten HS256-Ablauf.
- Später für Overex: wiederverwendbarer Workflow, damit alle Services ihr Image gleich bauen und pushen.

Commit-Verlauf

Datum	Hash	Nachricht
2026-10-09 23:52	`5c35460`	build(docker): mount the JWT keys into the backend container
2026-10-10 14:56	`e317fde`	ci(docker): build the backend image on every push to a non-main branch
2026-10-10 19:41	`8641c70`	ci: add backend-verify-ci and docker-publish workflows
2026-10-10 19:41	`0a1ea83`	docs(docker): comment every instruction of the Dockerfile and docker-ci
2026-10-10 20:03	`edd0e90`	Merge pull request #1 from Umex10/feature/docker

Anhang: Grundlagen ausführlich

Dieser Anhang fasst zusammen, was ich in diesem Meilenstein verstanden habe. Er ist so geschrieben, dass ich später (und für die Bachelorarbeit) darauf aufbauen kann.

A. Java, Maven und Spring Boot

A1. Vom Quellcode zur Jar

Der Java-Compiler `javac` übersetzt `.java`-Dateien in `.class`-Dateien. Das ist **Bytecode**, den die JVM ausführt. Eine **Jar** (Java ARchive) ist eine ZIP-Datei mit `.class`-Dateien, Ressourcen (zum Beispiel `application.properties`) und einer Beschreibungsdatei, dem **Manifest**. Es sagt, welche Klasse beim Start zuerst läuft.

A2. Maven im Vergleich mit npm

Aufgabe	npm-Projekt (Nest/Next)	AuthKit (Maven)
Projektbeschreibung	`package.json`	`pom.xml`
Abhängigkeiten laden	`npm ci`	Maven
Code übersetzen	`tsc` / `nest build` / `next build`	Maven ruft `javac` auf
Tests	`npm test` (Jest)	Maven ruft JUnit auf
Herkunft der Pakete	npm-Registry	Maven Central
Ablage der Pakete	`node_modules/` **im Projekt**	`~/.m2/repository/` **zentral für alle Projekte**
Build-Ergebnis	`dist/` bzw. `.next/`	`target/`
feste Tool-Version	`"packageManager"` / `.nvmrc`	`./mvnw` mit `.mvn/wrapper/maven-wrapper.properties`

Maven gibt eine feste Ordnerstruktur vor: `src/main/java`, `src/main/resources`, `src/test/java`, `target/`. Darum muss man Maven nicht sagen, wo der Code liegt.

A3. `<dependencies>` und `<build><plugins>`

- **Dependencies** sind Bibliotheken, die mein Code mit `import` benutzt (Spring Security, jjwt). Sie landen in der Jar.
- **Plugins** sind Werkzeuge, die Maven beim Bauen benutzt (Compiler, Spring-Boot-Plugin, JaCoCo). Mein Code sieht sie nie.

Lombok steht an beiden Stellen: als Dependency, weil mein Code `@Data` importiert, und beim Compiler-Plugin, weil der Compiler Lombok beim Übersetzen ausführen muss.

A4. Woher Maven weiß, was eine Bibliothek braucht

Zu jeder Bibliothek wird neben der Jar eine eigene `.pom`-Datei veröffentlicht, zum Beispiel:

```
~/.m2/repository/io/jsonwebtoken/jjwt-api/0.11.5/
├── jjwt-api-0.11.5.jar     ← die Bibliothek
└── jjwt-api-0.11.5.pom     ← ihre Abhängigkeiten
```

Maven liest diese `.pom`-Dateien rekursiv. So werden aus 20 Einträgen in meinem `pom.xml` 106 Bibliotheken in der fertigen App.

A5. Klassenpfad

`import org.springframework.security.authentication.AuthenticationManager;` ist eigentlich ein Pfad: `org/springframework/security/authentication/AuthenticationManager.class`. Java sucht ihn in einer Liste von Ordnern und Jars, dem **Klassenpfad**. Beim Kompilieren baut Maven diese Liste aus `~/.m2` (über 150 Einträge). Beim Start der fertigen App baut das Startprogramm von Spring Boot sie aus `BOOT-INF/lib/`.

A6. Die Maven-Phasen

```
validate → compile → test → package → verify → install → deploy
```

Wer eine Phase aufruft, bekommt alle davor automatisch mit:

```
./mvnw compile   -> validate, compile                      → target/classes (.class + Properties)
./mvnw test      -> ..., compile, test                     → target/surefire-reports
./mvnw package   -> ..., compile, test, package            → Jar in target/
./mvnw verify    -> ..., package, verify                   → bei mir zusätzlich der JaCoCo-Report
./mvnw install   -> ..., verify, install                   → Jar in mein lokales ~/.m2
./mvnw deploy    -> ..., install, deploy                   → Jar in eine entfernte Registry (wie npm publish)
./mvnw clean     -> löscht nur target/ (eigener Ablauf, steht oft davor: clean verify)
```

Befehle mit Doppelpunkt sind **Aufgaben eines Plugins**, keine Phasen. Sie laufen allein, ohne die Phasen davor:

```
./mvnw dependency:go-offline   -> lädt alle Abhängigkeiten und Plugins nach ~/.m2, baut nichts
./mvnw jacoco:report           -> erzeugt nur den Coverage-Report
./mvnw spring-boot:run         -> kompiliert und startet die App direkt (für die Entwicklung)
```

Wichtige Zusätze:

Zusatz	Bedeutung
`-B`	Batch-Modus: keine Rückfragen, keine Fortschrittsbalken. Für Docker und CI.
`-q`	quiet: nur Fehler ausgeben
`-DskipTests`	Tests kompilieren, aber nicht ausführen
`-Dmaven.test.skip=true`	Tests weder kompilieren noch ausführen
`-Dtest=KeyConfigTest`	nur diesen Test ausführen
`-Dtest='!com/authkit/backend/integration/**'`	alle Tests **außer** denen in `integration/`

A7. `target/`

`target/` ist der Arbeitsordner von Maven. Alles, was beim Bauen entsteht, liegt dort: `classes/`, `test-classes/`, `surefire-reports/`, `jacoco.exec`, `site/jacoco/` und die Jars. Er lässt sich jederzeit neu erzeugen, deshalb steht er in `.gitignore` und `.dockerignore`.

A8. `./mvnw` und `mvn`

`mvn` ist ein auf dem Rechner installiertes Maven (bei mir `/usr/bin/mvn`, Version 3.8.7). `./mvnw` ist ein Skript im Repo. Es liest `.mvn/wrapper/maven-wrapper.properties`:

```
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.14/apache-maven-3.9.1
```

Fehlt diese Version, lädt das Skript sie nach `~/.m2/wrapper/dists/` und startet sie dann mit meinen Argumenten. Den Java-Teil lädt es nicht: Ein JDK muss vorhanden sein.

Das `./` vor `mvnw` heißt „diese Datei im aktuellen Ordner“. Ohne `./` sucht Linux Programme nur in den Ordnern aus `PATH` (zum Beispiel `/usr/bin`), nicht im aktuellen Ordner. Das ist eine Sicherheitsmaßnahme.

A9. Schlanke Jar und Fat Jar

Bei `./mvnw package` zeigt das Log die Reihenfolge:

```
--- compiler:compile          ← .java → .class
--- surefire:test             ← Tests
--- jar:jar                   ← schlanke Jar bauen
--- spring-boot:repackage     ← Spring Boot ersetzt sie durch die Fat Jar
    "... with repackaged archive, adding nested dependencies in BOOT-INF/."
```

Maven allein packt **keine** Abhängigkeiten in die Jar, weil eine Maven-Jar als Bibliothek gedacht ist. Die schlanke Jar bleibt als `*.jar.original` liegen. Startet man sie, fehlen die Bibliotheken:

```
java -cp backend-0.0.1-SNAPSHOT.jar.original com.authkit.backend.BackendApplication
→ NoClassDefFoundError: org/springframework/boot/SpringApplication
```

Die Fat Jar (66 MB) enthält:

```
backend-0.0.1-SNAPSHOT.jar
├── BOOT-INF/classes/      ← meine 33 Klassen + application.properties
├── BOOT-INF/lib/          ← 106 Bibliotheks-Jars (Kopien aus ~/.m2)
├── org/springframework/boot/loader/   ← Startprogramm (JarLauncher)
└── META-INF/MANIFEST.MF   ← Main-Class: JarLauncher, Start-Class: BackendApplication
```

`java -jar app.jar` startet zuerst den `JarLauncher`. Er baut den Klassenpfad aus `BOOT-INF/lib` und ruft dann `BackendApplication.main()` auf. Darum braucht die laufende App kein `~/.m2`.

A10. JDK, JRE, JVM

```
JDK  =  JRE + Compiler (javac) und weitere Entwicklerwerkzeuge   → zum Bauen
JRE  =  JVM + Standardbibliothek + der Befehl java              → zum Ausführen
JVM  =  das Programm, das Bytecode ausführt
```

Vergleich mit Node: Zum Ausführen eines Nest-Backends reicht `node` (wie das JRE). Zum Bauen braucht man zusätzlich den TypeScript-Compiler (wie das JDK).

B. Docker

B1. Begriffe

- **Image**: unveränderliche Vorlage aus übereinanderliegenden Dateisystem-Schichten (Layers). Es läuft nicht.
- **Container**: laufender Prozess, der aus einem Image gestartet wurde.
- **Dockerfile**: das Rezept, aus dem ein Image gebaut wird.
- **Registry**: Server, auf dem Images liegen (Docker Hub, `ghcr.io`).

B2. Der Kernel und Container unter Linux

Der **Kernel** ist das Herzstück eines Betriebssystems und das einzige Programm, das direkt mit der Hardware spricht. Andere Programme bitten ihn um Dateien, Speicher oder Netzwerk. Linux ist streng genommen nur der Kernel, Linux Mint und Ubuntu sind Distributionen (Kernel plus Programme). Windows und macOS haben eigene Kernels.

Ein Container ist unter Linux ein **normaler Prozess**, dem der Kernel eine veränderte Sicht gibt:

- **Namespaces**: eigenes Dateisystem (nur die Schichten des Images), eigene Prozessliste (der Prozess hält sich für Nummer 1), eigenes Netzwerk
- **cgroups**: Grenzen für CPU und Arbeitsspeicher

Alle Container teilen sich den Kernel des Rechners. Ein Image enthält die Programme eines Ubuntu, aber keinen Kernel. Darum starten Container in Sekunden. Windows und macOS haben keinen Linux-Kernel, deshalb startet Docker Desktop dort eine Linux-VM (unter Windows über WSL2). Bei mir läuft die Docker Engine direkt auf Linux Mint, ohne VM.

B3. `docker`, `dockerd`, `containerd`, `runc`

```
docker (Befehl im Terminal)   nur ein Bote
   │  über /var/run/docker.sock
   ▼
dockerd                     Docker-Dienst (Daemon), läuft dauerhaft mit root-Rechten,
   │                        verwaltet Images, Container, Netzwerke, Volumes
   ▼
containerd                  startet, stoppt und überwacht Container
   │
   ▼
runc                        richtet die Abschottung im Kernel ein und startet den Prozess
```

Ein **Daemon** ist ein Programm, das im Hintergrund läuft und auf Aufträge wartet (wie `sshd`). **Docker Engine** ist alles außer dem Befehl `docker`. Kubernetes braucht `dockerd` nicht, es spricht direkt mit `containerd`. Das Image-Format ist ein Standard (OCI), deshalb läuft ein mit Docker gebautes Image dort unverändert.

Sicherheit: Wer Befehle an `dockerd` schicken darf (Gruppe `docker`), kann sich praktisch root-Rechte verschaffen. Risiken entstehen vor allem durch eingebundene Ordner, fremde Images und root im Container.

B4. Wie ein Image gebaut wird

1. `docker build` schickt den **Build-Kontext** (den Ordner `apps/backend` ohne die Dateien aus `.dockerignore`) an `dockerd`. Nur diese Dateien kann `COPY` holen.
2. `FROM` lädt das Basis-Image (falls nicht vorhanden) und nimmt seine Schichten als Grundlage. Es startet nichts.
3. `COPY` legt Dateien als neue Schicht ab, ohne Container.
4. Jede `RUN`-Zeile startet einen **kurzlebigen Container**, der alle bisherigen Schichten sieht und eine leere, beschreibbare Schicht obendrauf bekommt. Was der Befehl schreibt, wird als neue Schicht gespeichert, danach wird der Container entfernt.
5. Das fertige Image ist eine Liste von Schichten plus Einstellungen (`USER`, `EXPOSE`, `ENTRYPOINT`). Es läuft erst, wenn jemand es startet.

```
beschreibbare Schicht     ← hier schreibt der RUN-Befehl
──────────────────────
Schicht: COPY src         ← nur lesen
Schicht: RUN go-offline   ← nur lesen
Schicht: COPY pom.xml     ← nur lesen
Schichten: eclipse-temurin (Ubuntu + JDK)
──────────────────────
= das, was der Prozess als "/" sieht
```

B5. Die `Dockerfile` von AuthKit

```dockerfile
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN chmod +x mvnw && ./mvnw -B -q dependency:go-offline
COPY src ./src
RUN ./mvnw -B -q clean package -DskipTests

FROM eclipse-temurin:21-jre AS runtime
WORKDIR /app
RUN useradd --system --uid 1001 authkit
USER authkit
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Zeile	Bedeutung
`FROM eclipse-temurin:21-jdk AS build`	Basis-Image mit JDK 21 (Anbieter Eclipse Temurin), Stage mit dem Namen `build`
`WORKDIR /app`	Arbeitsordner anlegen und hineinwechseln
`COPY .mvn/ .mvn/`, `COPY mvnw pom.xml ./`	nur das, was zum Laden der Abhängigkeiten nötig ist
`RUN chmod +x mvnw && ./mvnw -B -q dependency:go-offline`	`mvnw` ausführbar machen (auch root braucht ein `x`-Bit, und unter Windows kann es verloren gehen), dann nur die Abhängigkeiten laden
`COPY src ./src`	den Code kopieren
`RUN ./mvnw -B -q clean package -DskipTests`	Fat Jar bauen, Tests nur kompilieren (sie laufen in der CI)
`FROM eclipse-temurin:21-jre AS runtime`	neue, frische Stage mit nur einem JRE; JDK, Maven und `.m2` bleiben zurück
`RUN useradd --system --uid 1001 authkit`, `USER authkit`	App läuft nicht als root, sondern als Dienstkonto mit fester uid (root wäre uid 0)

`COPY --from=build /app/target/*.jar app.jar`	nur die Fat Jar holen; `*.jar` erfasst die `.jar.original` nicht
`EXPOSE 8080`	nur Dokumentation; geöffnet wird der Port erst beim Start (`ports:`)
`ENTRYPOINT ["java", "-jar", "app.jar"]`	Startbefehl des Containers; in eckigen Klammern läuft `java` direkt als Hauptprozess und bekommt Stopp-Signale

Ergebnis: 396 MB, davon etwa 330 MB Basis-Image und 66 MB Jar.

Die `Dockerfile` ist **in sich geschlossen**: Ein Container sieht nur die Dateien seines Images, nie das Java auf meinem Rechner. Darum muss das JDK im Image sein, solange die Jar im Container gebaut wird. Dafür braucht man zum Bauen nur Docker und den Code.

B6. Der Layer-Cache und der Trick mit `go-offline`

Docker prüft jede Anweisung einzeln:

- bei `RUN` : Ist der **Befehlstext** gleich?
- bei `COPY` : Ist der **Inhalt der kopierten Dateien** gleich?

Ab der ersten Änderung wird alles darunter neu gebaut. Kommentare in der `Dockerfile` zählen nicht, die `Dockerfile` selbst wird nicht ins Image kopiert. Kommentare in kopierten Dateien (zum Beispiel in `src/`) zählen dagegen.

Ohne `go-offline` stünde das Herunterladen in der Zeile mit `package` , also **nach** `COPY src` . Ändert sich eine Java-Datei, läuft diese Zeile neu und startet auf dem Stand der Zeile davor, in dem noch kein gefülltes `.m2` liegt. Alles würde neu geladen. Mit `go-offline` liegt das gefüllte `.m2` in einer Schicht **über** `COPY src` , die im Cache bleibt, solange sich das `pom.xml` nicht ändert.

```
npm:    COPY package.json   →  RUN npm ci                       (nur herunterladen)
        COPY src            →  RUN npm run build                (bauen)
Maven:  COPY pom.xml        →  RUN ./mvnw dependency:go-offline (nur herunterladen)
        COPY src            →  RUN ./mvnw package               (bauen)
```

B7. `RUN` , `CMD` und `ENTRYPOINT`

`RUN` läuft beim **Bauen**. `ENTRYPOINT` und `CMD` legen fest, was beim **Start** läuft. Docker setzt den Startbefehl so zusammen:

```
ENTRYPOINT  +  (was beim Start hinter dem Image-Namen steht, sonst CMD)
```

Beispiel:

```
ENTRYPOINT ["ping"]
CMD ["google.com"]
```

`docker run image` führt `ping google.com` aus, `docker run image github.com` führt `ping github.com` aus. `ENTRYPOINT` bleibt immer stehen, `CMD` ist nur der Standard. Basis-Images bringen eigene Standardwerte mit (das `node` -Image zum Beispiel `CMD ["node"]`). Ein eigenes `ENTRYPOINT` verwirft auch ein geerbtes `CMD` . In Kubernetes ersetzt `command` das `ENTRYPOINT` und `args` das `CMD` .

B8. Image-Namen und Registries

```
docker.io / library / eclipse-temurin : 21-jdk
Registry    Bereich   Name              Tag

ghcr.io   / umex10  / authkit-backend   : latest
Registry    Besitzer  Name              Tag
```

Ohne Registry nimmt Docker Docker Hub, `library` ist der Bereich für offizielle Images. Tags wie `21-jre` wandern mit: Sie zeigen immer auf die neueste Java-21-Version, wechseln aber nie auf 22. Image-Namen dürfen nur Kleinbuchstaben enthalten.

B9. Bind Mount und benanntes Volume

```
services:
  db:
    volumes:
```

```
      - authkit_db_data:/var/lib/postgresql/data   # links ein Name → benanntes Volume
  backend:
    volumes:
      - ./apps/backend/keys:/app/keys:ro           # links ein Pfad → Bind Mount, nur lesend

volumes:
  authkit_db_data:                                 # Anmeldung des benannten Volumes
```

	Bind Mount	**benanntes Volume**
Ort	ein Ordner, den ich wähle	Docker verwaltet ihn (`/var/lib/docker/volumes/`)
Anmeldung unten in `volumes:`	nicht nötig	nötig
gut für	Dateien, die ich mitgebe (Keys)	Daten, die der Container erzeugt (Datenbank)
Ordner fehlt	Docker legt ihn **leer** an, ohne Fehler (Tippfehler fallen erst in der App auf)	Docker legt das Volume an

Bei einem Bind Mount sind es dieselben Dateien: Schreibt der Container hinein, ändern sich die Dateien auf dem Rechner. Darum `:ro` . Compose setzt vor benannte Volumes den Projektnamen (`authkit_authkit_db_data`). Ein Volume überlebt `docker compose down` , erst `docker compose down -v` löscht es.

B10. Keys und Rechte

Das Image enthält nie Geheimnisse, denn es liegt in einer Registry. Die Keys kommen beim Start von außen: mit Compose per Bind Mount, mit `docker run` per `-v` , in Kubernetes per **Secret**. Fehlen sie, bricht die App sofort mit klarer Meldung ab („fail fast“). In Kubernetes müssen alle Kopien des Service denselben privaten Key bekommen, und ein neuer Key macht alle alten Tokens ungültig.

Linux-Rechte bestehen aus drei Bits (`r` =4, `w` =2, `x` =1) für drei Gruppen (Besitzer, Gruppe, alle anderen):

```
private.pem vorher   rw- --- ---   = 600   nur ich
private.pem nachher  rw- r-- r--   = 644   ich lese und schreibe, alle anderen lesen
```

Der Container läuft als uid 1001, für Linux ein fremder Benutzer, und brauchte deshalb das Leserecht für „alle anderen“.

B11. `docker compose`

`docker compose up --build backend` baut das Image neu und startet `backend` samt `db` (wegen `depends_on`). Compose legt ein gemeinsames Netzwerk an (`authkit_default`), in dem sich die Container über ihre Service-Namen erreichen. Darum funktioniert `jdbc:postgresql://db:5432/authkit` .

C. GitHub Actions

C1. Aufbau einer Workflow-Datei

Workflows liegen in `.github/workflows/` im Hauptordner des Repos. Jede `.yml` -Datei ist ein Workflow.

Baustein	**Bedeutung**
`name`	Anzeigename
`on`	**wann** der Workflow läuft (Push, Pull Request, Knopfdruck)
`jobs`	Aufgaben; jeder Job läuft auf einer **eigenen, frischen VM** (`runs-on: ubuntu-latest`), die am Ende gelöscht wird
`steps`	Schritte eines Jobs, nacheinander
`run`	ein Befehl im Terminal der VM; ich beschreibe den Schritt selbst
`uses`	eine fertige **Action**; jemand anderes hat den Schritt beschrieben
`with`	Einstellungen für eine Action, wie Parameter einer Methode

Auf der VM ist am Anfang nichts: kein Code, kein `.m2`, kein Docker-Cache. Docker ist aber vorinstalliert. Darum ist der erste Schritt fast immer `actions/checkout`, der den auslösenden Commit klont.

C2. Actions

`actions/checkout@v7` heißt: Organisation `actions` (gehört GitHub), Repository `checkout`, Version `v7`. `docker/...` gehört der Organisation von Docker, die beiden Organisationen liegen nebeneinander. Jede Action ist ein normales Repository mit einer `action.yml` (Eingaben, Ausgaben, was ausgeführt wird) und dem Code dazu.

Versionen: Jede Veröffentlichung bekommt einen festen Tag (`v7.0.1`). Der Tag `v7` wird auf die neueste 7er-Version verschoben, wechselt aber nie auf 8. Am strengsten ist eine volle Commit-Nummer. Manche Actions werden zusätzlich als unveränderliches Paket veröffentlicht und erscheinen dann unter „Packages".

Eine **eigene Action** (zum Beispiel eine Composite Action aus einer Liste von Schritten) ist ein wiederverwendbarer Schritt. Ein **wiederverwendbarer Workflow** ist ein ganzer Workflow, den andere als Job aufrufen. Beides liegt in einem normalen Repo, im eigenen Konto oder in einer Organisation.

C3. Die drei Workflows und wann sie laufen

Workflow	Auslöser	Macht
`backend-verify-ci`	jeder Push mit Änderungen im Backend, alle Branches **inkl.** **`main`**	`./mvnw verify`, Reports als Artifact
`docker-ci`	jeder Push mit Änderungen im Backend, **außer** `main`	Image bauen, nicht pushen
`docker-publish`	Push auf **`main`**, oder per Knopf	Image bauen und nach `ghcr.io` pushen

Bei einem Push startet GitHub alle Workflows, deren Auslöser passt, parallel und unabhängig voneinander. Der Auslöser gilt pro Workflow. Soll ein Job auf einen anderen warten (`needs`), müssen beide im selben Workflow stehen.

Ein PR zeigt die Ergebnisse der Läufe, die für seinen neuesten Commit schon gelaufen sind. Neue Läufe startet er nur mit dem Auslöser `pull_request`, der dann den Stand nach dem Merge prüft.

C4. `docker-ci.yml` (ohne Kommentare)

```yaml
name: docker-ci
on:
  push:
    branches-ignore:
      - main
    paths:
      - "apps/backend/**"
      - ".github/workflows/docker-ci.yml"
permissions:
  contents: read
concurrency:
  group: docker-ci-${{ github.ref }}
  cancel-in-progress: true
jobs:
  build-image:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v7
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v4
      - name: Build backend image
        uses: docker/build-push-action@v7
        with:
          context: apps/backend
          push: false
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

Teil	Bedeutung
`branches-ignore: main`	alle Branches außer `main`
`paths`	nur starten, wenn sich das Backend oder diese Datei ändert; `**` heißt „alles darin, auch Unterordner“. Jede Änderung zählt, auch ein Kommentar.
`permissions: contents: read`	der automatische Token darf nur den Code lesen
`concurrency.group`	Name einer Warteschlange. Gleicher Name: Läufe beeinflussen sich. Mit `github.ref` (Branch) hat jeder Branch seine eigene, mit dem Präfix jeder Workflow.
`cancel-in-progress: true`	ein neuer Lauf bricht den laufenden des älteren Commits ab
`setup-buildx-action`	richtet nur das Werkzeug ein: einen Builder, der mit externem Cache umgehen kann
`build-push-action`	baut, liest den Cache (`cache-from`), schreibt ihn zurück (`cache-to`) und pusht, wenn `push: true`
`context: apps/backend`	der Build-Kontext; die `Dockerfile` darin wird benutzt
`mode=max`	speichert auch die Schichten der Build-Stage (mit den Maven-Abhängigkeiten)

Die Maschine braucht hier kein Java: Die Jar entsteht in einem Container aus dem JDK-Image.

C5. `docker-publish.yml` (ohne Kommentare)

```yaml
name: docker-publish
on:
  push:
    branches:
      - main
    paths:
      - "apps/backend/**"
      - ".github/workflows/docker-publish.yml"
  workflow_dispatch:
permissions:
  contents: read
  packages: write
concurrency:
  group: docker-publish
  cancel-in-progress: false
jobs:
  publish-image:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v7
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v4
      - name: Log in to ghcr.io
        uses: docker/login-action@v4
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - name: Image metadata
        id: meta
        uses: docker/metadata-action@v6
        with:
          images: ghcr.io/${{ github.repository_owner }}/authkit-backend
          tags: |
            type=raw,value=latest,enable={{is_default_branch}}
            type=sha
      - name: Build and push backend image
        uses: docker/build-push-action@v7
```

```
        with:
          context: apps/backend
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

Teil	Bedeutung
`workflow_dispatch`	Knopf „Run workflow“ im Reiter „Actions“, zum Beispiel um nach einem Sicherheitsupdate des Basis-Images neu zu veröffentlichen
`packages: write`	der Token darf in GitHub Packages (also `ghcr.io`) schreiben
`GITHUB_TOKEN`	automatischer Zugangsschlüssel für genau diesen Lauf, liegt in `secrets`; Werte aus `secrets` erscheinen im Log als `***`
`github.actor`	wer den Lauf ausgelöst hat
`cancel-in-progress: false`	neue Läufe warten statt abzubrechen
`metadata-action`	**berechnet** nur Text: Image-Namen in Kleinbuchstaben, Tags nach den Regeln unter `tags`, und automatisch Labels
`type=raw,value=latest,...`	fester Tag `latest`, nur auf dem Haupt-Branch
`type=sha`	Tag `sha-` plus die ersten 7 Zeichen der Commit-Nummer
`id: meta`, `steps.meta.outputs.tags`	späterer Schritt liest das Ergebnis dieses Schritts
`labels`	unter anderem `org.opencontainers.image.source`; darüber verknüpft GitHub das Paket mit dem Repository

Nach `tags: |` ist alles reiner Text, auch Zeilen mit `#`. Kommentare gehören darüber.

Ein Image, mehrere Namen: `metadata-action` liefert

```
ghcr.io/umex10/authkit-backend:latest
ghcr.io/umex10/authkit-backend:sha-<commit>
```

`build-push-action` baut das Image **einmal** und pusht es unter beiden Namen. `latest` wandert mit, `sha-...` zeigt für immer auf genau einen Stand. Für Kubernetes nimmt man den festen Tag.

C6. `backend-verify-ci.yml` (ohne Kommentare)

```
name: backend-verify-ci
on:
  push:
    paths:
      - "apps/backend/**"
      - ".github/workflows/backend-verify-ci.yml"
  workflow_dispatch:
permissions:
  contents: read
concurrency:
  group: backend-verify-ci-${{ github.ref }}
  cancel-in-progress: true
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v7
      - name: Set up JDK 21
        uses: actions/setup-java@v6
        with:
```

```
          distribution: temurin
          java-version: "21"
          cache: maven
          cache-dependency-path: apps/backend/pom.xml
      - name: Test and build (./mvnw verify)
        working-directory: apps/backend
        run: ./mvnw -B verify -Dtest='!com/authkit/backend/integration/**'
      - name: Upload test and coverage reports
        if: ${{ !cancelled() }}
        uses: actions/upload-artifact@v7
        with:
          name: backend-reports
          path: |
            apps/backend/target/surefire-reports/
            apps/backend/target/site/jacoco/
          if-no-files-found: ignore
          retention-days: 14
```

Teil	Bedeutung
kein Branch-Filter	Tests laufen überall, auch nach dem Merge auf `main`
`setup-java`	installiert JDK 21 **direkt auf der VM**, weil Maven hier ohne Docker läuft
`distribution: temurin` , `java-version: "21"`	dasselbe JDK wie im Image; Anführungszeichen, damit YAML die Zahl als Text liest
`cache: maven`	lädt `~/.m2/repository` beim Start aus dem GitHub-Cache und am Ende wieder hoch
`cache-dependency-path`	der Fingerabdruck des `pom.xml` bestimmt den Cache-Eintrag; neue Abhängigkeit, neuer Eintrag
`working-directory`	der Befehl läuft in `apps/backend` statt im Hauptordner
`-Dtest='!…/integration/**'`	vorübergehend alle Tests außer den Integrationstests
`if: ${{ !cancelled() }}`	ein Schritt läuft standardmäßig nur, wenn alle davor erfolgreich waren. So läuft der Upload auch bei roten Tests, aber nicht nach einem Abbruch.
`upload-artifact`	hängt Dateien als ZIP an den Lauf (unten auf der Seite des Laufs)
`if-no-files-found: ignore`	`warn` (Standard) gäbe eine Warnung, `error` würde den Schritt rot machen, `ignore` schweigt
`retention-days: 14`	GitHub löscht das Artifact nach 14 Tagen

C7. GitHub-Cache und Artifacts

Der **GitHub-Cache** liegt auf den Servern von GitHub und gehört einem **Repository** (nicht dem Konto). Pro Repository gibt es 10 GB für alle Branches und Workflows zusammen. Einträge, die sieben Tage nicht benutzt werden, werden gelöscht. Ein Branch sieht seinen eigenen Cache und den von `main` , nicht den anderer Feature-Branches. Unter „Actions → Caches“ sieht man die Einträge, zum Beispiel:

- `buildkit-blob-1-sha256:...` : eine Schicht; der Name ist ein Fingerabdruck des Inhalts, gleicher Inhalt wird nur einmal gespeichert
- `index-buildkit-1-...` : das Inhaltsverzeichnis; Docker lädt es zuerst und holt dann nur die passenden Schichten
- `setup-java-...-maven-...` : der Maven-Ordner aus `backend-verify-ci`

Nach dem ersten Lauf von `docker-ci` lagen 19 Einträge im Cache (Basis-Image-Schichten wegen `mode=max` , eigene Schichten, Inhaltsverzeichnis). Anweisungen wie `EXPOSE` oder `USER` erzeugen keine Schicht mit Dateien.

```
Start des Laufs:   GitHub-Cache  ──herunterladen──►  Maschine (~/.m2 bzw. Builder)
Ende des Laufs:    GitHub-Cache  ◄──hochladen──────  Maschine
```

	Cache	Artifact
für wen	spätere Läufe (schneller)	mich (ansehen, herunterladen)

gehört zu	dem Repository	einem bestimmten Lauf
herunterladen	nicht vorgesehen	ja, als ZIP

C8. Registries und GitHub Packages

Eine **Registry** ist ein Server, der Pakete **eines Formats** aufbewahrt und ausgibt (abgeben: `publish` / `push` , holen: `install` / `pull`). Für jedes Format gibt es mehrere: für Docker zum Beispiel Docker Hub und `ghcr.io` , für npm die öffentliche npm-Registry und die von GitHub, für Maven Maven Central.

GitHub Packages bietet mehrere Registries unter einem Dach (npm, Maven, Docker). `ghcr.io` (GitHub Container Registry) ist die Docker-Registry darin. Pakete gehören einem **Konto oder einer Organisation**. Der Reiter „Packages“ zeigt alle Pakete aus allen GitHub-Registries in einer Liste. Über das Label `org.opencontainers.image.source` wird ein Paket mit einem Repository verknüpft und erscheint dort in der Seitenleiste.

C9. Merge-Strategien

Strategie	**Ergebnis**	**gut, wenn**
Create a merge commit	alle Commits unverändert, plus ein Merge-Commit	die Commits sauber sind und man sehen will, was zu einem PR gehörte
Squash and merge	alle Commits zu einem zusammengefasst	der Branch viele Zwischenstände enthält
Rebase and merge	Commits einzeln in gerader Linie, aber mit **neuen** Hashes	man eine gerade Historie will und sich nicht auf Hashes bezieht

Ich habe für PR #1 den Merge-Commit gewählt, weil meine Meilenstein-Doku auf Commit-Hashes verweist.